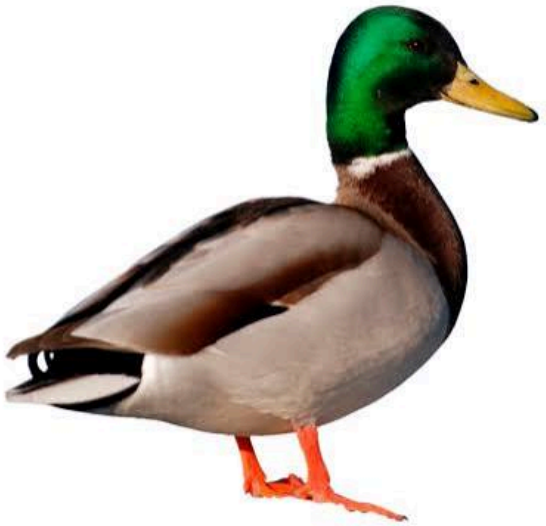
Ciência de Dados Aplicada ao Direito I

Aula 17

Funções em lote e classes



Prof. Bruno da Cunha de Oliveira - 2026/2

A função dentro do for

Insper

Chamar a função dentro do for

```
def prazo_em_dobro(dias):
    return dias * 2

prazos = [15, 5]
dobrados = []
for simples in prazos:
    dobrado = prazo_em_dobro(simples)
    dobrados.append(dobrado)
print(dobrados)
```

A parte tem prazo em dobro em dois prazos. O que aparece na tela?

A função dentro do for, passo a passo

Animação no navegador: `animacao-lote.html`, cenário `prazos`.

Uma chamada por volta

RESPOSTA

```
[30, 10]
```

A linha da chamada está escrita uma vez, e o `for` a repete. Na primeira volta, `dias` recebe 15; na segunda, 5. Aqui o `return` devolve a conta direto, sem guardar o resultado numa variável antes.

O mesmo resultado sem função

```
prazos = [15, 5]
dobrados = []
for simples in prazos:
    dobrados.append(simples * 2)
print(dobrados)
```

Este programa também mostra `[30, 10]`, e não tem função. Em que situação a versão com função passa a valer a pena?

Quando a função vale a pena

RESPOSTA

Com a conta numa linha e num lugar só, tanto faz.

A função vale quando	O que ela resolve
a regra aparece em mais de um lugar	mudou a regra, corrige-se num lugar só
a regra tem `if` e `else` dentro	o `for` continua curto e legível
a regra precisa ser conferida	testa-se um caso antes de rodar em mil

A triagem de dois casos de estelionato

```
def configura_estelionato(vantagem, prejuizo, erro, fraude):
    return vantagem and prejuizo and erro and fraude

casos = [{"investigado": "Marina Alves", "vantagem": True,
          "prejuizo": True, "erro": True, "fraude": False},
         {"investigado": "Otávio Bastos", "vantagem": True,
          "prejuizo": True, "erro": True, "fraude": True}]
```

Escrevam o `for` que aplica a função a cada caso e guarda as respostas na lista `conclusoes`.

Os campos do caso viram os argumentos

RESPOSTA

```
conclusoes = []
for caso in casos:
    conclusao = configura_estelionato(
        caso["vantagem"], caso["prejuizo"],
        caso["erro"], caso["fraude"])
    conclusoes.append(conclusao)
```

`conclusoes` fica com `[False, True]`. A função é a mesma de um caso só, e quem repete é o `for`.

Uma função que preenche um modelo

Preencher dois campos do modelo

```
modelo = "Manifeste-se <PARTE> em <DIAS> dias."

def preencher(modelo, parte, dias):
    modelo.replace("<PARTE>", parte)
    modelo.replace("<DIAS>", str(dias))
    return modelo

print(preencher(modelo, "a autora", 30))
```

O que aparece na tela?

Guardar o que o replace devolve

RESPOSTA

```
Manifeste-se <PARTE> em <DIAS> dias.
```

`.replace()` devolve um texto novo e não altera `modelo`. A função jogou fora os dois textos e devolveu o modelo como chegou.

```
    texto = modelo.replace("<PARTE>", parte)
    texto = texto.replace("<DIAS>", str(dias))
    return texto
```

A segunda troca parte de `texto`, o resultado da primeira.

No PrairieLearn: exercício 6

Insper

Função que chama função

Dois passos, duas funções

```
def prazo_em_dobro(dias):
    return dias * 2

def dias_restantes(prazo, decorridos):
    return prazo - decorridos

total = prazo_em_dobro(15)
faltam = dias_restantes(total, 12)
print(faltam)
```

Apelação: 15 dias, em dobro, com 12 já decorridos. O que aparece na tela?

Uma função depois da outra, passo a passo

Animação no navegador: `animacao-encadeamento.html`, cenário `sequencia`.

A saída de uma é a entrada da outra

RESPOSTA

```
18
```

`prazo_em_dobro` devolveu 30, que ficou guardado em `total`. `total` entrou como argumento de `dias_restantes`, e 30 menos 12 dá 18.

Uma função para os dois passos

```
def prazo_em_dobro(dias):
    return dias * 2
```

Escrevam a função `restantes_em_dobro`. Ela recebe `dias` e `decorridos` e devolve quantos dias restam de um prazo em dobro.

Ela deve chamar `prazo_em_dobro`, sem repetir a conta do dobro.

Uma função dentro da outra, passo a passo

Animação no navegador: `animacao-encadeamento.html`, cenário `dentro`.

Uma função chama a outra

RESPOSTA

```
def restantes_em_dobro(dias, decorridos):
    prazo = prazo_em_dobro(dias)
    return prazo - decorridos

faltam = restantes_em_dobro(15, 12)
```

`restantes_em_dobro` chama `prazo_em_dobro` e espera a resposta. `faltam` guarda 18, e a regra do dobro continua escrita num lugar só.

O for dentro de uma função

```
def prazo_em_dobro(dias):
    return dias * 2
```

Escrevam a função `dobrar_todos`. Ela recebe uma lista de prazos e devolve a lista dos prazos em dobro, na mesma ordem.

Ela deve chamar `prazo_em_dobro` para cada prazo.

A função do lote chama a função do item

RESPOSTA

```
def dobrar_todos(prazos):
    dobrados = []
    for simples in prazos:
        dobrados.append(prazo_em_dobro(simples))
    return dobrados
```

`dobrar_todos([15, 5])` devolve `[30, 10]`. O `return` fica fora do `for`: a função só devolve depois da última volta.

Um resultado diferente do esperado

```
def prazo_em_dobro(dias):
    return dias * 2

def restantes_em_dobro(dias, decorridos):
    prazo = prazo_em_dobro(dias)
    return decorridos - prazo

print(restantes_em_dobro(15, 12))
```

Eu esperava 18, e apareceu −18. Que chamada vocês fariam para descobrir em qual das duas funções está o defeito?

Conferir cada função sozinha

RESPOSTA

```
print(prazo_em_dobro(15))
```

```
30
```

Sozinha, `prazo_em_dobro` responde certo. O defeito está em `restantes_em_dobro`: a conta saiu invertida, `decorridos - prazo`. Conferir cada função sozinha diz onde procurar.

A ordem dos argumentos

Insper

Ler uma chamada com quatro argumentos

```
def configura_estelionato(vantagem, prejuizo, erro, fraude):
    return vantagem and prejuizo and erro and fraude

resposta = configura_estelionato(True, True, False, True)
```

Qual dos quatro fatos não foi apurado neste caso? Como vocês descobriram?

Argumento nomeado

RESPOSTA

Foi `erro`, o terceiro. Para saber, é preciso contar a posição e voltar à definição.

```
resposta = configura_estelionato(
    vantagem=True, prejuizo=True,
    erro=False, fraude=True)
```

Cada argumento pode ir com o nome do parâmetro. A chamada passa a dizer os fatos.

Argumentos nomeados em outra ordem

```python
def dias_restantes(prazo, decorridos):
    return prazo - decorridos

print(dias_restantes(30, 12))
print(dias_restantes(decorridos=12, prazo=30))
print(dias_restantes(12, 30))
```

O que aparece na tela?

Com nome, a ordem não importa

RESPOSTA

```
18
18
-18
```

Sem nome, vale a posição, e a terceira chamada trocou o prazo e os dias decorridos. Com nome, cada valor vai para o parâmetro que a chamada indica, em qualquer ordem.

Menos argumentos que parâmetros

Uma chamada com um argumento a menos

```python
def dentro_do_teto(valor_sm, teto=40):
    return valor_sm <= teto

print(dentro_do_teto(50))
print(dentro_do_teto(50, teto=60))
```

A definição tem dois parâmetros, e a primeira chamada traz um argumento só. O que aparece na tela?

Parâmetro opcional

RESPOSTA

```
False
True
```

`teto=40` na definição dá ao parâmetro um valor padrão: o teto do Juizado Especial Cível. Sem o segundo argumento, a causa de 50 salários mínimos passa do teto. No Juizado Especial Federal o teto é de 60, e a mesma função responde: basta escrever `teto=60`.

Um programa de verdade

Insper

Um programa com o juscraper

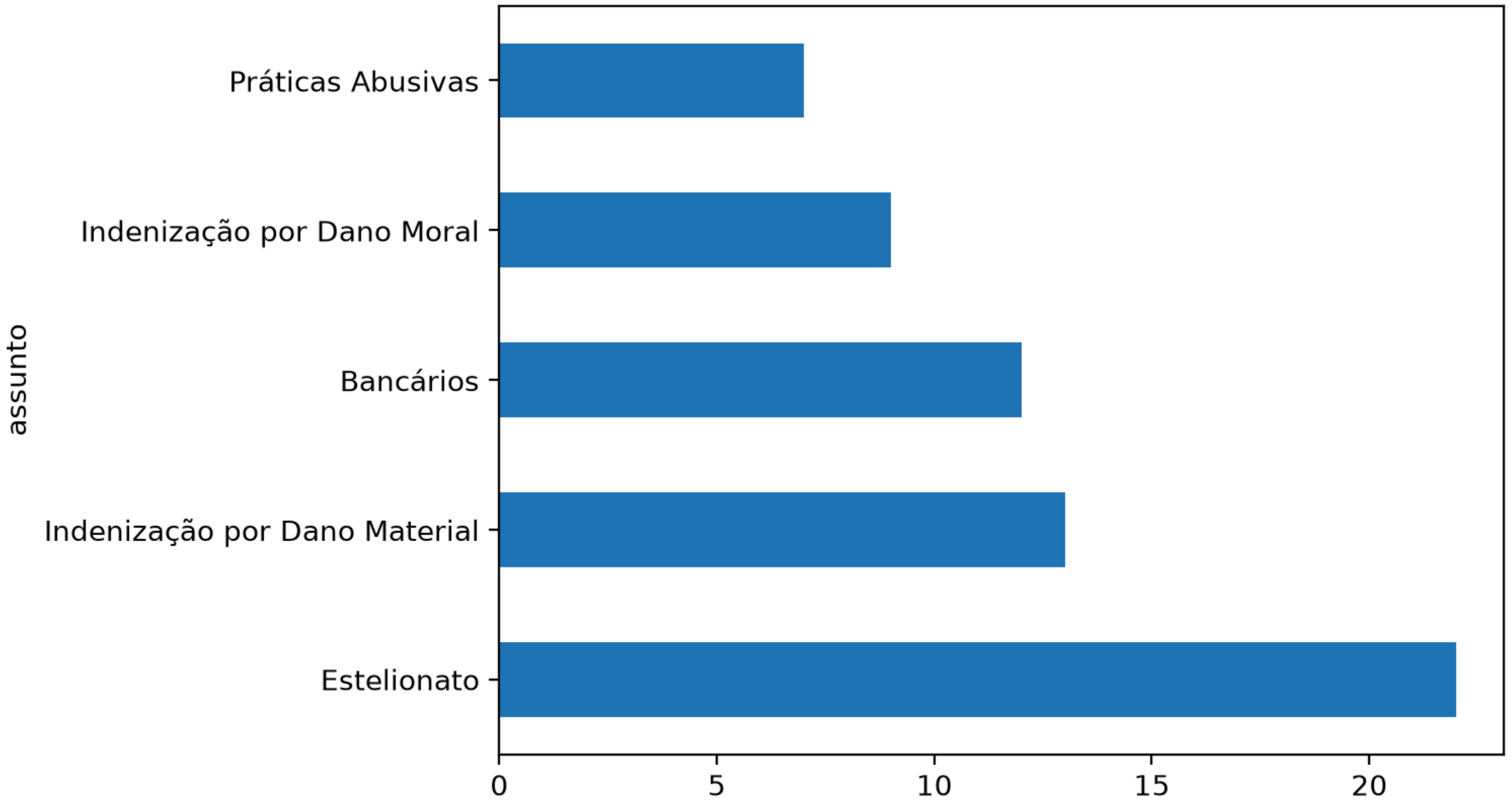
Leiam sozinhos. Que partes vocês já entendem, e que partes ainda são mistério?

```
import juscraper as jus

tjsp = jus.scraper("tjsp")
dados = tjsp.cjpg("golpe do pix", paginas=range(1, 11))

dados.shape
contagem = dados["assunto"].value_counts()
contagem.head(5).plot(kind="barh")
```

O gráfico que esse programa produz



range(1, 4) num for

```
for pagina in range(1, 4):
    print(pagina)
```

O que aparece na tela?

range: até antes do segundo número

RESPOSTA

```
1
2
3
```

`range(1, 4)` entrega 1, 2 e 3: começa no primeiro número e para antes do segundo. No programa, `paginas=range(1, 11)` pede as páginas de 1 a 10, e `paginas=` é um argumento nomeado.


SONY
Discman
GAME BOY COLOR
TYCO
TYCO
27MHz


Oct 28, Oct 28
10:09

Python não vem com pilha

O que vem e o que não vem com o Python

VEM COM O PYTHON

Texto, número, lista e dicionário. `if`, `for` e funções.

É tudo o que vocês usaram até agora.

O que vem e o que não vem com o Python

VEM COM O PYTHON

Texto, número, lista e dicionário. `if`, `for` e funções.

É tudo o que vocês usaram até agora.

NÃO VEM

Tabelas. Gráficos. Leitura de PDF. Coleta de decisões em tribunais. Modelos de linguagem.

É quase tudo o que um trabalho com dados precisa.

Bibliotecas

O que não vem com o Python se instala.

São as bibliotecas: código que outra pessoa escreveu e publicou. O juscraper é uma delas.

O ponto depois do valor

Insper

O ponto depois de um texto

```
tribunal = "TJSP"
print(tribunal.lower())
print(tribunal)
```

O que aparece na tela?

Um método: o texto sabe virar minúsculas

RESPOSTA

```
tjsp
TJSP
```

O ponto pede uma ação ao valor que vem antes dele: `tribunal.lower()` devolve o texto em minúsculas. Como o `.replace()`, ele devolve um texto novo, e `tribunal` continua como estava. Essa ação se chama método.

Métodos de texto

Chamada	Devolve	O que faz
`"TJSP".lower()`	`tjsp`	passa para minúsculas
`"tjsp".upper()`	`TJSP`	passa para maiúsculas
`"  TJSP  ".strip()`	`TJSP`	tira os espaços das pontas
`"TJSP".replace("SP", "RJ")`	`TJRJ`	troca um trecho por outro

Vocês já usavam métodos: `.replace()` é do texto, e `.append()`, da lista.

O ponto depois de uma lista

```
pecas = ["petição inicial", "contestação"]
pecas.append("sentença")
print(pecas)
print(pecas.lower())
```

O que aparece na tela?

Cada tipo tem os seus métodos

RESPOSTA

```
['petição inicial', 'contestação', 'sentença']
AttributeError: 'list' object has no attribute 'lower'
```

`.append()` é método de lista, e `.lower()`, de texto. Uma lista não sabe virar minúsculas, e o Python diz isso no erro: a lista não tem `lower`.

Função ou método

Elemento	Como se usa	Exemplos
função	vive solta, e o valor entra entre os parênteses	`len(pecas)`, `print(pecas)`
método	mora dentro de um tipo, e age sobre o valor antes do ponto	`pecas.append("sentença")`, `tribunal.lower()`

Um método é uma função que já sabe sobre qual valor vai agir.

Criar um tipo

O molde e os cachorros

A CLASSE É O MOLDE

```
CLASSE Cachorro:
    nome
    idade
    latir()
```

CADA CACHORRO É UM OBJETO

```
rex:  nome "Rex",  idade 3
toto: nome "Totó", idade 8
```

O molde é escrito uma vez. Cada cachorro sai dele com os seus valores. Texto e lista são tipos que alguém escreveu assim, e dá para escrever os nossos.

class e `__init__`

```python
class Cachorro:
    def __init__(self, nome, idade):
```

`class` cria o tipo e dá o nome a ele: é o molde.

`__init__` é uma função como as que vocês já escrevem, com um nome reservado. O Python a chama sozinho, toda vez que um cachorro é criado: é ela que preenche a ficha dele.

self: este cachorro

```
class Cachorro:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade
```

`self` é o cachorro que está sendo criado.

`self.nome = nome` guarda neste cachorro o nome que chegou pela chamada.
`self.idade = idade` faz o mesmo com a idade.

Criar dois cachorros

```
class Cachorro:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

rex = Cachorro("Rex", 3)
toto = Cachorro("Totó", 8)
print(rex.idade, toto.idade)
```

O que aparece na tela?

Atributo: o que cada objeto guarda

RESPOSTA

```
3 8
```

`Cachorro("Rex", 3)` chama o `__init__`: `"Rex"` vai para `nome`, e 3, para `idade`. Quem chama não escreve o `self`. `rex.idade` lê o que ficou guardado no Rex: é um atributo, e cada cachorro tem os seus.

Um cachorro que late

```
class Cachorro:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    def latir(self):
        return self.nome + ": au au"
toto = Cachorro("Totó", 8)
print(toto.latir())
```

Acrescentei uma função dentro da classe. O que aparece na tela?

Método: uma função dentro da classe

RESPOSTA

```
Totó: au au
```

Uma função escrita dentro da classe é um método. Em `toto.latir()`, `self` é o `toto`, e `self.nome` vale `"Totó"`. É o mesmo ponto de `tribunal.lower()`: pede uma ação ao objeto.

Atributo e método

```
toto = Cachorro("Totó", 8)
```

Elemento	O que é	Neste caso
atributo	um dado que o objeto guarda; sem parênteses	`toto.idade`
método	uma ação que o objeto executa; com parênteses	`toto.latir()`

A ficha da parte

```python
class Parte:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    def tem_prioridade(self):
        return self.idade >= 60
```

Na autuação, alguém preenche a ficha de cada parte. A classe é a ficha em branco, o `__init__` é o preenchimento, e cada parte do processo é uma ficha preenchida.

Criar uma parte, passo a passo

Animação no navegador: `animacao-objetos.html`, cenário `criar`.

Escrever a classe Processo

Escrevam a classe `Processo`, com dois atributos e um método.

Os atributos são `numero`, o número do processo, e `valor_sm`, o valor da causa em salários mínimos. O método `dentro_do_teto()` devolve `True` quando o valor não passa de 40 salários mínimos, o teto de valor do Juizado Especial Cível, e `False` quando passa.

Vale 0,1 ponto extra na prova final para quem entregar a classe pelo formulário "classe Processo", no Blackboard, até o fim da aula, com as duas respostas do próximo slide certas.

Conferir antes de entregar

Depois da classe de vocês, rodem estas três linhas:

```
causa = Processo("0000123-27.2024.8.99.0001", 15)
outra = Processo("0000456-76.2024.8.99.0001", 50)
print(causa.dentro_do_teto(), outra.dentro_do_teto())
```

A classe está pronta para entregar quando aparecer:

```
True False
```

A classe Processo

RESPOSTA

```python
class Processo:
    def __init__(self, numero, valor_sm):
        self.numero = numero
        self.valor_sm = valor_sm

    def dentro_do_teto(self):
        return self.valor_sm <= 40
```

A comparação já vale `True` ou `False`, e o `return` a entrega.

O programa do juscraper, de novo

```
import juscraper as jus

tjsp = jus.scraper("tjsp")
dados = tjsp.cjpg("golpe do pix", paginas=range(1, 11))

dados.shape
contagem = dados["assunto"].value_counts()
contagem.head(5).plot(kind="barh")
```

Leiam de novo. O que deixou de ser mistério?

O que já dá para ler

RESPOSTA

No programa	Agora vocês leem como
`tjsp`, `dados`, `contagem`	objetos, cada um de uma classe
`.cjpg(...)`, `.head(5)`	métodos: o ponto pede uma ação
`dados.shape`	atributo: um dado, sem parênteses
`paginas=range(1, 11)`	argumento nomeado; páginas de 1 a 10

Ainda é mistério: o `import`, os colchetes de `dados["assunto"]` e de que classe são esses objetos.

No PrairieLearn: Desafio V

Insper